CLASE 8 · 3 HS

Máquinas de Estado e Integrador

Del diagrama de estados al código con enum y switch, y el **Entrenador Lógico** completo: todo el curso en un menú.

Programación en C con ESP32 · Sistemas Lógicos · Universidad Nacional de Santiago del Estero · 2026

HOY

Cuatro bloques, un entrenador completo

1 · 15 min

enum y switch

2 · 45 min

Máquinas de estado: del diagrama al código

3 · 90 min

Integrador: Mi Entrenador Lógico (trabajo asistido)

4 · 30 min

Ensayo de defensa en parejas

El entrenador **no es un programa nuevo**: es pegar con un menú lo que ya hicieron, de a un modo por vez.



enum: ponerle nombre a los números

```
enum Estado { VERDE, AMARILLO, ROJO };
// VERDE vale 0, AMARILLO 1, ROJO 2

Estado estado = VERDE;  // de tipo Estado

if (estado == ROJO) {   // se lee solo
  digitalWrite(LEDS[0], HIGH);
}
```

ANTES

`if (estado == 2)`

¿Qué es el 2? ¿Rojo? ¿Abierta? ¿Alarma? Es un **número mágico**: nadie lo sabe dentro de dos semanas.

Conexión con la teoría

Es la **asignación de estados** (VERDE = 00, AMARILLO = 01, ROJO = 10): la hace el compilador y nosotros usamos los nombres.

Tres trucos que usa el entrenador

```
enum Modo { MODO_A, MODO_B, MODO_C, CANTIDAD_MODOS };
```

El truco del último: el valor extra al final es la cantidad. CANTIDAD_MODOS vale 3.

```
modo = (Modo)((modo + 1) % CANTIDAD_MODOS);  // 0 → 1 → 2 → 0
```

Pasar al siguiente: el % vuelve al primero después del último; (Modo) promete que el entero es un Modo válido.

```
enum Boton { BOT_A, BOT_B, BOT_C };  →  seApreto(BOT_A)
```

La promesa de la Clase 7: es lo mismo que seApreto(0), pero se lee solo. No es BOTON_A porque ese ya es el pin 32.

switch: elegir entre muchos caminos

```
switch (estado) {
  case VERDE:
    Serial.printf("Pasen\n");
    break;       // termina el switch
  case ROJO:
    Serial.printf("Alto\n");
    break;
  default:     // ningún case coincidió
    Serial.printf("Estado raro\n");
    break;
}
```

`switch (variable)`

Mira el valor: entero o enum.

`case VALOR:`

"Si vale esto, empezar aquí".

`break;`

Salta afuera del switch.

`default:`

Si ningún case coincidió. Opcional, pero recomendado.

El error más famoso: olvidar el break

```
switch (modo) {
  case 0:
    Serial.printf("BINARIO\n");
    // ¡falta el break!
  case 1:
    Serial.printf("COMPUERTAS\n");
    break;
}
```

Sigue de largo

Con modo == 0 imprime **BINARIO y COMPUERTAS**. Un case es solo una etiqueta de "empezar aquí".

Compila sin errores y funciona mal. Excepción: un return dentro del case.



switch para una variable contra valores fijos; **if** para condiciones de varias variables, rangos o tiempos. En una máquina de estados se usan los dos.

Una máquina de estados: el semáforo

Siempre en **uno** de pocos estados · cada estado tiene sus **salidas** · cambia por entrada, evento o tiempo.



Como con flip-flops: la variable **estado** es el registro, el **switch** es la lógica combinacional y cada vuelta de **loop()** es un pulso de reloj.

La receta: de diagrama a C en 5 pasos

Paso	En el diagrama	En C
1	Cada **circulito**	`enum Estado { VERDE, AMARILLO, ROJO };`
2	La flecha de **inicio**	`Estado estado = VERDE;`
3	Cada circulito	`un case en switch (estado), con break`
4	Lo de **adentro** (salidas)	`digitalWrite, mostrarByte…`
5	Cada **flecha que sale**	`if (condición) { estado = OTRO; }`

Con tiempos: **inicioEstado** guarda cuándo se entró al estado, y "pasaron 5 s" es **millis() - inicioEstado >= 5000** (Clase 7).

El semáforo en C, sin un solo delay

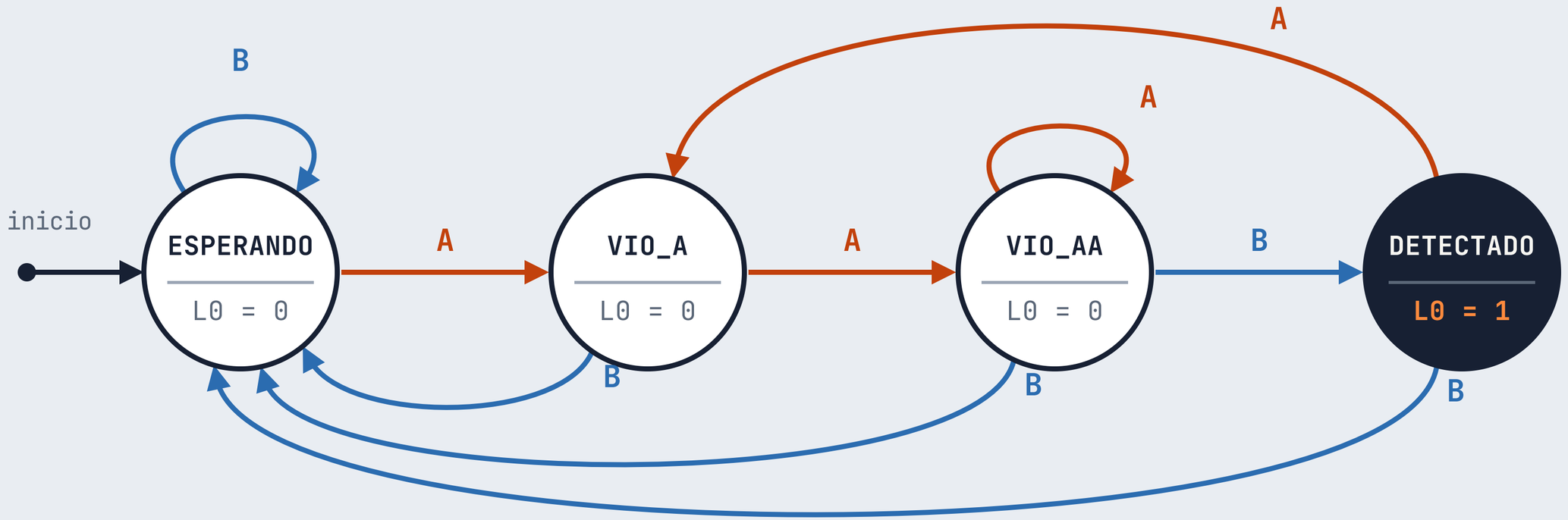
```
void loop() {
  unsigned long enEstado = millis() - inicioEstado;
  switch (estado) {
    case VERDE:
      luces(false, false, true);      // salida
      if (enEstado ≥ TIEMPO_VERDE) { // flecha
        cambiarEstado(AMARILLO);
      }
      break;
    // ... AMARILLO (2 s) y ROJO (5 s), igual
  }
}
```

Toda transición pasa por **cambiarEstado()**: cambia el estado y reinicia **inicioEstado = millis()**.

Para pensar

¿Y si se hace con **delay(5000)**? Funciona... hasta que se quiera agregar un botón.

Detector de A, A, B



Cada estado tiene una flecha **A** y una **B**. **VIO_AA + A** se queda (A A A); **DETECTADO + A → VIO_A** (superpuestas).

Moore o Mealy: ¿dónde va la salida?

Moore: en el estado

Depende **solo del estado**. Se dibuja dentro del circulito (ESTADO/salida) y dura mientras dure el estado. Suele necesitar más estados.

```
// afuera de las transiciones
digitalWrite(LEDS[0], estado == DETECTADO);
```

Mealy: en la flecha

Depende del **estado y de la entrada**. Se dibuja sobre la flecha (entrada/salida) y dura un instante. Alcanzan 3 estados.

```
case VIO_AA:
  if (flancoB) {
    Serial.printf("DETECTADO!\n");
    estado = ESPERANDO;
  }
  break;
```

La cerradura: estado y datos

Estado	Salida	Sale cuando…
BLOQUEADA	L0 fijo	A o B → INGRESANDO
INGRESANDO	L0 + barra verde con las teclas	4 teclas → ABIERTA, BLOQUEADA o ALARMA; 5 s sin teclas → BLOQUEADA
ABIERTA	L4..L7	5 s, o botón C → BLOQUEADA
ALARMA	L0..L3 titilando	10 s → BLOQUEADA

El estado dice **qué** hace la cerradura; teclas, claveBien e intentosFallidos dicen **cuánto** lleva.

En hardware serían **4 × 3 × 5 = 60 estados**. En software: 4 estados y unas variables.

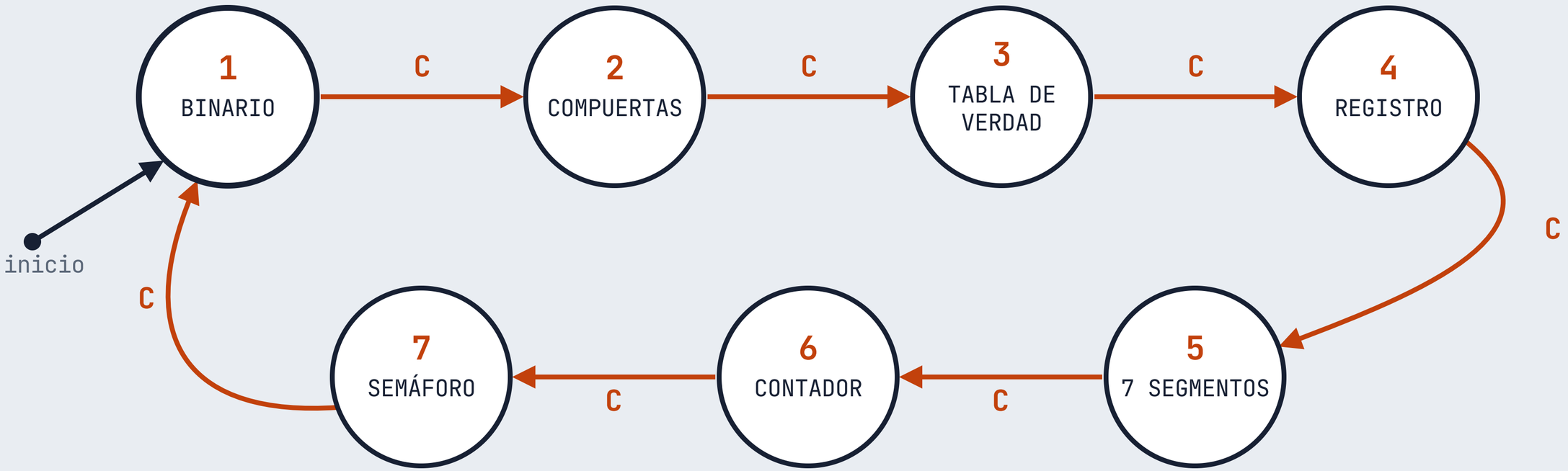
3 · INTEGRADOR · PASO 0

Planificar los modos en papel

#	Modo	Viene del TP	A	B	LEDs
1	BINARIO	Clase 2	pausa / sigue	vuelve a 0	el contador en binario
2	COMPUERTAS	Clases 3 y 4	entrada A	entrada B	una compuerta por LED
3	TABLA DE VERDAD	Clase 4	compuerta siguiente	anterior	número de compuerta
4	REGISTRO	Clase 5	rotar	invertir	el registro
5	7 SEGMENTOS	Clase 6	dígito +1	dígito −1	los segmentos
6	CONTADOR	Clase 7	+1	−1	el contador
7	SEMÁFORO / CERRADURA	Clase 8	según la FSM		las luces

El menú también es una FSM: `1 BINARIO` **C** → `2 COMPUERTAS` **C** → … `7 SEMÁFORO` **C** → vuelve al 1

El menú también es una máquina de estados



Cada flanco de **C**: `modo = (Modo)((modo + 1) % CANTIDAD_MODOS);` A y B no cambian de modo.

El esqueleto que compila

```
void loop() {
  bool flancoA = seApreto(BOT_A);   // UNA vez, aquí arriba
  bool flancoB = seApreto(BOT_B);
  bool flancoC = seApreto(BOT_C);
  if (flancoC) {
    entrarModo((Modo)((modo + 1) % CANTIDAD_MODOS));
  }
  switch (modo) {
    case MODO_BINARIO:  modoBinario(flancoA, flancoB);  break;
    case MODO_CONTADOR: modoContador(flancoA, flancoB); break;
  }
}
```

El loop() **no hace casi nada**: lee entradas, atiende el menú y le pasa el control al modo activo.

Primero, **modos vacíos**: que compile y que C cambie de modo. Después, el contenido.



Traer los modos de a uno

En el TP había…	En el entrenador va…
void loop() { … }	**void modoContador(bool flancoA, bool flancoB)**
if (seApreto(0)) dentro del loop()	**if (flancoA)**: seApreto ya se llamó una vez
delay(500); para ir más lento	**if (millis() - ultimoPaso >= 500)**
Variables **static** que recuerdan	Globales con nombre claro, para reiniciarlas al entrar
pinMode en el setup() del TP	Una sola vez, en el setup() del entrenador
Prints en el loop() sin condición	Prints solo cuando algo cambia

Convertir un TP en función de modo, **probarlo**, y recién ahí pasar al siguiente. Nunca pegar tres modos juntos sin probar.

Entrar a un modo: anunciar y reiniciar

```
void entrarModo(Modo nuevo) {
  modo = nuevo;
  inicioModo = millis();   // empieza el cartel
  Serial.printf("\n===== Modo %d: ", modo + 1);
  switch (modo) {
    case MODO_CONTADOR:
      Serial.printf("CONTADOR (A: +1, B: -1) =====\n");
      contador = 0;   // reiniciar SUS variables
      break;
    // ... un case por modo
  }
}
```

1. Imprime el nombre del modo y qué hacen A y B.
2. **Reinicia** las variables de ese modo.
3. Muestra el número de modo en los LEDs: un cartel de 800 ms, **sin delay**.

Las reglas de oro del entrenador

Regla	Por qué
Los botones se leen **una sola vez**, al principio del loop()	Si dos funciones llaman a seApreto(BOT_A), la primera se come el flanco y la segunda nunca lo ve
Ningún delay dentro de un modo	Congela el menú: C deja de responder
Cada case termina en **break**	Si no, se ejecuta también el modo de abajo
La variable modo empieza **inicializada**	Si no, el entrenador empieza en un modo cualquiera
Al entrar a un modo se **reinician** sus variables	Si no, el modo empieza "a la mitad"

⚠ El entrenador de la cátedra está en **referencia/**: para comparar organización, no para copiar. En la defensa hay que explicar cada línea.

EJERCICIOS · CANDIDATOS AL MODO 7

Para practicar máquinas de estado

EJERCICIO 1

Semáforo peatonal

A es el pulsador del peatón: acorta el verde de 10 s a 3 s y enciende L2 ("ESPERE"). Mealy.

EJERCICIO 2

Detector de 1011

A = 0, B = 1. Diagrama de Moore (5 estados) y después el código. Con 1011011 detecta dos veces.

EJERCICIO 3

Máquina expendedora

Producto de $300. A = $100, B = $200, C = cancelar. Entrega y vuelto, o devolución.

BONUS

Dado electrónico

Con 7 segmentos. A tira: RODANDO 1,5 s con random(1, 7) y después MOSTRANDO.

Cualquiera puede ser **el modo 7** del entrenador. Soluciones en **referencia/**.

ENTREGA FINAL

Mi Entrenador Lógico

1. Un **menú** que se recorre con los pulsadores: C cambia de modo.
2. Al menos **4 modos** de los vistos en el curso.
3. Código organizado en **funciones**, sin bloques copiados y pegados.
4. Una **máquina de estados** con enum y switch.

ADEMÁS

- No se cuelga y responde siempre: nada de delay largos.
- Encabezado con nombre y cómo se usa cada botón.
- Prolijo: indentado, nombres claros, secciones en orden.
- Al cambiar de modo imprime el nombre y qué hacen A y B.

Entrega: carpeta **SL2026-Apellido-Entrenador** en ZIP, sin la carpeta .pio, antes de la fecha de defensa.



4 · ENSAYO DE DEFENSA

La defensa: 10 minutos en tres partes

3 min

Demo

Recorrer los modos con C y mostrar qué hacen A y B en cada uno, con un guion.

4 min

Explicación

El docente señala una parte del código: qué hace, por qué está y qué pasaría si se cambia.

3 min

Modificación en vivo

Un cambio chico, sin IA. Saber **dónde** y **cómo** tocar; no hace falta que salga perfecto.

Para aprobar hay que poder **explicar cada línea** del código propio. Hoy, ensayo en parejas: uno hace de docente y el otro defiende.

CHECKLIST DE SALIDA

Al terminar la clase, sé…

- declarar un enum y decir qué valor tiene cada nombre
- escribir un switch con case, break y default, y qué pasa sin break
- pasar un diagrama de estados a código: enum + switch + un if por flecha
- explicar Moore y Mealy, y dónde va la salida en C
- temporizar estados con millis(), sin delay
- tener el entrenador con menú y al menos 2 modos funcionando

Lo que viene: no hay Clase 9. Sigue **la defensa**: terminar los modos, entregar el ZIP y practicar con el banco de modificaciones de DEFENSA.md.